

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Duration :60 Mins
On class
Total Marks:50

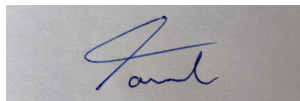
Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I Tamil Elakiya S (192524443) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler"

    int count;

    count = findLength();

    if(count > 5
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2
            printf("Option 2\n");
            break;

        default:
```

```

        printf("Invalid Choice\n");
    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}

```

Tasks

(a) Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)

sol

- **Line no** : 35 , **Error** : char name[20]="Compiler" ,**Type** : Syntax error , missing colon.
- **Line no** : 37 , **Error** : findlength(); , **Type** : semantic error , Function requires a argument , argument is missing.
- **Line no** : 38 , **Error** : if(count>5 , **Type** : Syntax error ,) parenthesis is missing .
- **Line no** : 43 , **Error** : marks[4] , **Type** : Runtime error , the index is out of range .
- **Line no** : 50 , **Error** : case 2 , **Type** : Syntax error , Missing colon.
- **Line no** : 57-58 , **Error** : *ptr , **Type** : Runtime error , Pointer not initialized .
- **Line no** : 61 , **Error** : total/students , **Type** : Runtime error , Division by Zero .
- **Line no** : 63-66 , **Error** : while(i<=5), **Type** : Logical error , Infinite loop .. i value is not incremented.

(b) Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)

Sol

- Syntax error : missing colon.
- semantic error : Function requires a argument , argument is missing.
- Syntax error ,) parenthesis is missing .
- Runtime error , the index is out of range .
- Syntax error , Missing colon.
- Runtime error , Pointer not initialized .
- Runtime error , Division by Zero .
- Logical error , Infinite loop .. i value is not incremented.

(c) Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)

Sol

- **Lexical** - Lexical Analysis
- **Syntax** - Syntax Analysis
- **Semantic** - Semantic Analysis
- **Logical** - not detected by compiler, observed during execution
- **Runtime** - occurs during execution.

(d) Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)

#Code :

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler";

    int count;

    count = findLength(name);

    if(count > 5)
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[3]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2:
```

```

        printf("Option 2\n");
        break;

    default:
        printf("Invalid Choice\n");
    }
    int value = 50;
    int *ptr = &value;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 5;

    float average = (float)total / students;

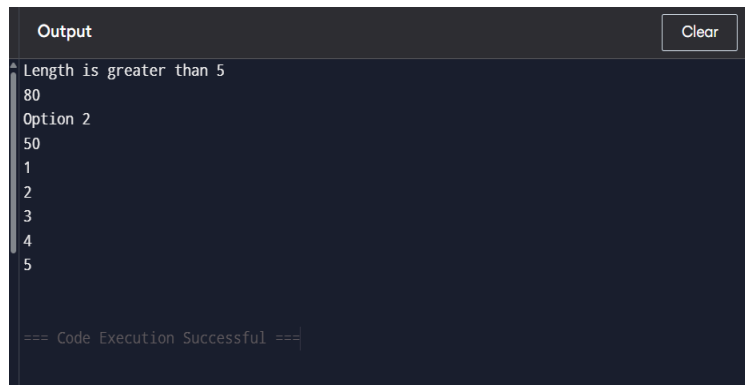
    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}

```

// OutPut :



The screenshot shows a dark-themed window titled "Output" with a "Clear" button in the top right corner. The output text is as follows:

```

Length is greater than 5
80
Option 2
50
1
2
3
4
5

=== Code Execution Successful ===

```

(e) Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

Sol :

1. Syntax Error Checking :

Detects missing brackets , semicolons and colons.

2. Type checking :

Detects incorrect function arguments and incompatible data types .

3. Compiler Warnings :

Warns about unused variables , array boundaries and possible division by zero.

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>

int maximum(int x, int y
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n")
            break;

        case 90
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
```

```

    max--;
while(max > 75);

if(max == 75)
    printf("Limit Reached\n"
}

return 0

```

Tasks

(a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)

Sol :

- **Line no : 3 , Error :** maximum(int x,int y , **Type :** Syntax error , missing).
- **Line no : 20 , Error :** printf(...) , **Type :** Syntax error , Missing semicolon.
- **Line no : 22 , Error :** case 90, **Type :** Syntax error , Missing colon.
- **Line no : 32 , Error :** while(max>75); , **Type :** Syntax error , Missing closing brace of do block before while.
- **Line no : 34 , Error :** printf("Limit Reached\n" , **Type :** Syntax error , Missing) and ;

(b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)

Sol:

- Missing) - Parentheses Error
- Missing ; - Statement Terminator Error
- Missing : - Switch-case Error
- Incorrect do-while syntax - Loop Structure Error
- Missing) and ; - Function Statement Error

(c) Correct the syntax errors and rewrite the complete program. (4 Marks)

Code :

```
#include <stdio.h>
```

```

int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}

```

```

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    }while(max > 75);

    if(max == 75)
    {
        printf("Limit Reached\n");
    }

    return 0;
}

```

// OutPut :

```

Maximum is 80
80
79
78
77
76
Limit Reached

```

```

=== Code Execution Successful ===

```


(d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)

Ans :

- The parser compares the source code with the grammar rules of the C language , when it encounters missing bracket , colon , or invalid statements it reports syntax error.
- Error Recovery methods such as Panic Mode Recovery , Phrase Level Recovery and Error Productions allow the compiler to continue parsing after detecting the error.

(e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

Ans :

- Intelligent error messages suggesting possible fixes.
- Error highlighting with exact line and column numbers.

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```
int factorial(int n)
```

```
{
```

```
    if(n <= 1)
```

```
        return 1;
```

```
    return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int @num = 5;
```

```
    int value = 10
```

```
    int result;
```

```

result = factorial();

if(result >= 100)
{
    printf("Large Number\n");
}

switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;

    case 120
        printf("Factorial of 5\n");
        break;

    default:
        printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[5]);

int *ptr;
printf("%d\n", *ptr);

int a = 50;
int b = 0;
int c = a / b;

int sum = result + 0 + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```

Tasks

(a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

Sol :

- **Line no** : 13 , **Error** : @num , **Type** : Lexical error , @ is an invalid identifier character.
- **Line no** : 15 , **Error** : Missing semicolon , **Type** : Syntax error , Statement not terminated.
- **Line no** : 19 , **Error** : factorial() , **Type** : Semantic error , Missing required argument.
- **Line no** : 31 , **Error** : case 120 , **Type** : Syntax error , Missing colon .
- **Line no** : 39 , **Error** : numbers[5] , **Type** : Runtime error , Array out of bounds .
- **Line no** : 42 , **Error** : *ptr , **Type** : Runtime error , Pointer not initialized .
- **Line no** : 47 , **Error** : a/b , **Type** : Runtime error , Division by Zero .
- **Line no** : 52 , **Error** : infinite loop , **Type** : Logical error , Infinite loop .. i value is not incremented.

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

Sol :

- Lexical Analysis - Invalid symbol @
- Syntax Analysis - Missing semicolon, missing colon
- Semantic Analysis - Function called without argument
- Intermediate Code Generation - Invalid syntax prevents intermediate code generation
- Code Optimization - Expression +0 is redundant
- Runtime - Array overflow, division by zero, uninitialized pointer, infinite loop

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

- Lexical Analysis : Converts characters into tokens , Detects invalid identifiers .
- Syntax Analysis : Validates program grammar.
- Semantic Analysis : Checks function calls and variable usage .
- Intermediate Code Generation : Generates intermediate representation only if previous phases are successful.
- Code Optimization : Removes unnecessary operations such as adding zero (+0).
- Runtime : Occurs during execution such as Division by zero , array overflow, etc.

(d) Rewrite the complete corrected program so that it compiles and executes without errors.

(2 Marks)

#Code :

```
#include <stdio.h>
```

```
int factorial(int n)
```

```
{
```

```
    if(n <= 1)
```

```
        return 1;
```

```
    return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int num = 5;
```

```
    int value = 10;
```

```
    int result = factorial(num);
```

```
    if(result >= 100)
```

```
    {
```

```
        printf("Large Number\n");
```

```
    }
```

```
    switch(result)
```

```
    {
```

```
        case 24:
```

```
            printf("Factorial of 4\n");
```

```
            break;
```

```
        case 120:
```

```
            printf("Factorial of 5\n");
```

```
            break;
```

```
        default:
```

```
            printf("Other Value\n");
```

```
    }
```

```
    int numbers[5] = {2,4,6,8,10};
```

```
    printf("%d\n", numbers[5]);
```

```

int x=20;
int *ptr=&x;
printf("%d\n", *ptr);

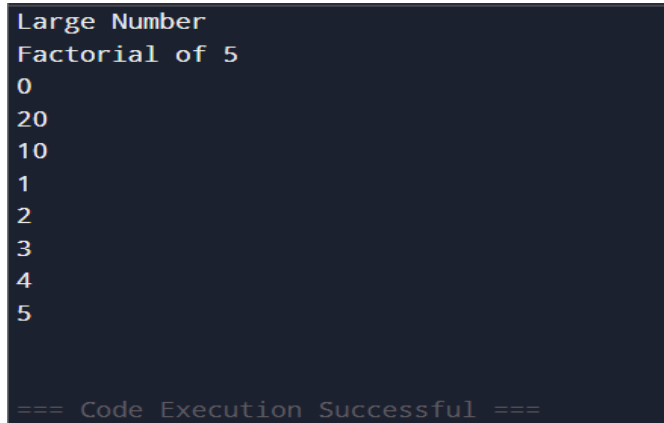
int a = 50;
int b = 5;
int c = a / b;
printf("%d\n", c);
int sum = result + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}

```

// OutPut :



```

Large Number
Factorial of 5
0
20
10
1
2
3
4
5
=== Code Execution Successful ===

```

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

- Detects null pointers, array bounds violations, and division-by-zero possibilities before execution.
- Removes redundant expressions and warns about unreachable code, helping produce faster and safer programs.